

Plan-Aware Prefetch for Agentic LLM Serving at the Edge

(Position Paper)

Hannah B. Pasandi, Tamer Nadeem

Submitted for double-blind review on IEEE/ACM SEC 26.

Abstract—Agentic LLM workloads are reshaping edge serving: a single edge node must absorb concurrent agents that spend most per-step latency on prefill, retrieval, and tool calls before generation begins. Existing edge serving stacks address this reactively or with weak signals, prefetching from developer-specified workflow graphs, semantic similarity to past queries, or coarse multi-turn hints. None reads what the agent itself already declares about its own next step. We argue that the agent’s reasoning trace is a prefetch oracle and propose PALSE, an edge serving middleware built on three primitives: a plan reader that streams declared next-step descriptors from the in-flight token stream; a plan-conditioned prefetch policy that stages KV state, retrieval embeddings, and tool outputs during the generation overlap window; and a speculative coherence protocol with token-identical-prefix salvage that bounds the wasted-work cost when plans are revised. We derive a coherence invariant and an analytical breakeven, report preliminary simulator and real-hardware evidence, and outline the experiments that would settle whether plan-aware prefetch holds in deployment.

I. INTRODUCTION

When demand on an edge node rises, the node cannot replicate to neighbouring sites the way cloud serving can [1]; it must absorb the contention or shed load. For agentic LLM workloads, which spend most of their end-to-end wall-clock in sequential dependencies between reasoning, retrieval, and tool calls, contention manifests as TTFT inflation on concurrent prefill, KV cache eviction, and queue stalls that cascade across multi-step trajectories. Recent measurement studies of on-device LLM inference [2] establish that mobile serving is energy- and latency-bounded; agentic workloads inherit these constraints and add a multi-step structure on top. The agentic-LLM serving problem is the latest instance of the long-running cloudlet problem [1]: limited local resources, a cloud that is round-trip-bounded, and a design question of what to put where and when.

LLM agents (ReAct [3], AutoGen [4], Gorilla [5]) now serve customer support [6], web navigation [7], and operating-system control [8]. Unlike a one-shot completion, an agent works through a problem in a sequence of steps, each emitting a thought, calling a tool or retriever, and continuing on the result. The latency a user perceives is the time to a useful answer at the end of a chain that may run a dozen steps deep. Each step pays a prefill cost on the accumulated context plus an auxiliary cost (retrieval, tool, RTT). On 2,000 traces across eight workloads (§II), these account for 78 percent of cloud-served per-step latency, with median TTFT of 260 ms and p95 of 535 ms.

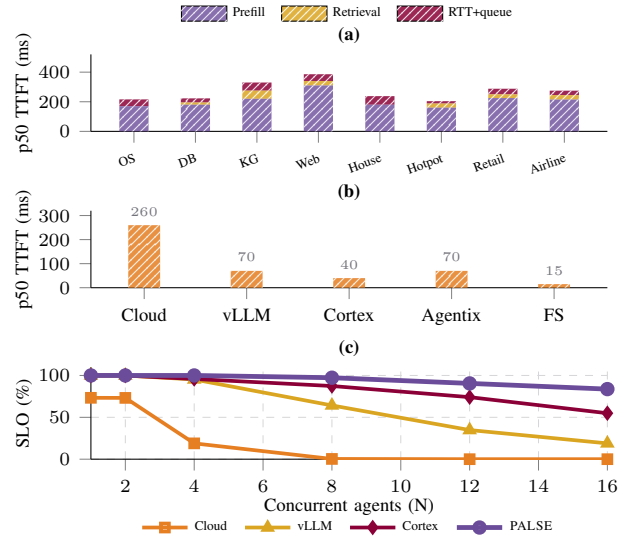


Fig. 1. Per-step latency breakdown (a), median TTFT by policy (b), and SLO under contention (c).

The agent already declares what it intends to do next: ReAct emits a *Thought* block before *Action*; planner-executor agents emit a multi-step plan; tool-augmented chat surfaces the function call as structured output. The serving system has the entire next-step intent in plain text before the agent commits to a tool call.

Modern LLM serving stacks address this contention reactively or with weak signals. vLLM [9] and SGLang [10] cache prefixes after the request arrives. KVFlow [11], the closest contemporary agentic-serving system, prefetches KV cache from a *developer-specified* Agent Step Graph encoding inter-agent dependencies. Cortex [12] prefetches retrieval state from semantic similarity to past queries. SYMPHONY [13] migrates KV caches across coarse multi-turn hints, and Agentix [14] intercepts program-issued LLM calls for priority scheduling without prefetch. None of these reads what the agent itself declares about its own next step at runtime, even though agentic frameworks [3], [15] emit that declaration explicitly before the corresponding action commits. All lose the 280 ms generation overlap window during which the next step’s state could have been staged speculatively.

The plan is the prefetch oracle. If the serving system reads the plan as the agent emits it, it can stage three things during

the overlap window: the KV state for the predicted next-step prefix, the retrieval embedding for the predicted search query, and the tool output for any deterministic tool the plan declares.

Across our eight calibrated workloads, agents change course between 8 and 22 percent of steps. A serving system that prefetches blindly burns proportional work on plans that never execute, and the wasted prefill state consumes the same edge GPU memory and prefill bandwidth that the actual next step will need. The mechanism only pays off under a coherence protocol that bounds the wasted-work cost.

We propose PALSE, a plan-aware prefetch layer for agentic LLM serving at the edge. PALSE runs as a middleware in front of vLLM and rests on three named primitives. The *plan reader* watches the in-flight token stream from the LLM and extracts the declared plan as soon as it is recognized, typically 200 to 300 ms before the agent emits the corresponding action token. The *plan-conditioned prefetch policy* stages KV state, retrieval embeddings, and tool outputs for the declared steps in step-distance order, bounded by the available overlap window. The *speculative coherence protocol* handles plan revision through partial-prefix salvage: when the agent’s actual next step diverges from the declared plan, prefetched state with at least 60 percent prefix overlap is reused as a partial KV cache and only the trailing tokens are reprefilled.

We evaluated PALSE on a calibrated five-policy comparison across 2,000 traces and 18,824 steps spanning AgentBench [8], ReAct on HotpotQA [3], [16], and τ -bench [6]. Median per-step TTFT drops from 40 ms (Cortex, the strongest baseline) to 15 ms; under 12-agent contention on a single edge node, PALSE holds 90.5 percent SLO where Cortex collapses to 74.1 percent. The implementation, simulator, trace corpus, and baseline reimplementations will be released as open source upon acceptance.

Broader impact. Moving agentic inference to the edge keeps reasoning traces and retrieved documents on-device, reducing cloud-side data exposure. The $2.4\times$ per-step energy reduction lowers operational cost, but speculative prefetch’s wasted-work overhead can erode carbon savings under adverse grid regimes (§V). Plan-aware prefetch also introduces a *future-behaviour* side channel that does not exist for purely reactive caches: an attacker co-located on the edge node can infer the victim agent’s next planned action, not just its past behaviour.

Scope. We focus on the single-edge-node setting; multi-tier coordination across the device–cloudlet–regional–cloud continuum is left as future work. Plan-aware tier placement, using the plan reader’s predicted resource trajectory to inform handoff decisions, is a natural extension.

This work makes the following contributions.

- Identifying the plan-as-prefetch-oracle observation, with a formal differentiation against four contemporary agentic-serving systems (KVFLOW, Cortex, Symphony, Agentix) on signal source and granularity (§III, §VI).
- Designing a three-primitive system, PALSE, with a coherence invariant on the salvage protocol and an analytical breakeven derivation (§III).

TABLE I
WORKLOAD PROFILES.

Workload	Steps	Revision	Retrieval
AgentBench OS [8]	8.4	14%	5%
AgentBench DB [8]	6.7	16%	10%
AgentBench KG [8]	9.1	13%	65%
AgentBench Web [8]	11.2	21%	30%
AgentBench Household [20]	13.7	18%	2%
ReAct/HotpotQA [3], [16]	4.2	8%	85%
τ -bench retail [6]	12.4	22%	45%
τ -bench airline [6]	9.1	18%	55%

- Evaluating the design via a calibrated simulator across eight agentic workloads (§IV), with a per-workload SLO breakdown, p99 tail-latency reporting, and cache-hit speedup validated against PyTorch transformer measurements at five operating points (mean -12 percent agreement).
- Outlining a six-experiment agenda spanning empirical fidelity, workload generality, sustainability profile, and failure-mode taxonomy that would settle the position empirically (§V).

II. BACKGROUND

A. Agentic LLM Workloads

An agentic LLM trajectory is a sequence of steps, each a single inference call preceded by some combination of retrieval and tool execution. Every step accumulates context from the prior step’s output plus any newly retrieved or observed information. The pattern appears in ReAct [3], plan-and-execute architectures [6], [15], multi-agent conversation frameworks [4], the broader family of tool-augmented agents surveyed by Patil *et al.* [5] and Qin *et al.* [17], and mobile agent deployments [18]. Multi-turn evaluation benchmarks [19] confirm that the structured plan-action-observation pattern is now standard.

In ReAct on HotpotQA the median trajectory is 4.2 steps with 85 percent of steps performing a single retrieval call. In AgentBench’s web environment trajectories average 11.2 steps with 30 percent retrieval rate and 21 percent of steps revising the plan from the previous step. In τ -bench’s retail environment trajectories average 12.4 steps with 22 percent revision rate, the highest in our corpus. The full per-workload profile appears in Table I and is calibrated against the public dataset distributions.

B. Per-Step Cost Decomposition

Each step incurs four costs: a prefill cost proportional to the accumulated context length, a retrieval cost when the step issues a retrieval call, a tool-execution cost when the step calls an external tool, and a network round-trip when the inference is cloud-served. For Llama-3.2-3B on a Jetson AGX-class edge node calibrated against vLLM benchmarks, these decompose as: prefill at 12.5 tokens per ms, retrieval at 18 ms fixed plus 0.04 ms per retrieved token, and tool latency drawn from the per-workload empirical distribution.

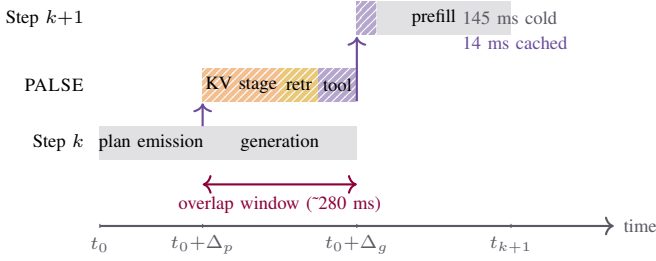


Fig. 2. Plan emission opens a generation overlap window for prefetch.

The prefill grows with each step. By step five of an AgentBench Web trajectory the prefill has grown to roughly 1,500 tokens, costing 120 ms at the calibrated edge rate before generation can begin. By step ten the prefill exceeds 3,000 tokens and prefill alone exceeds the 200 ms interactive SLO budget.

C. The Plan-Reading Opportunity

Production agent frameworks instruct the model to emit its plan before it acts [3], [15]. The plan appears as a bounded prefix of the model’s output, typically tagged with markers such as `Thought:`, `Plan:`, or `<thinking>`, and declares the next one to four steps in the trajectory. This declaration is produced before any tool invocation or retrieval call, giving a strict generation overlap window during which the serving system can stage state for the declared future steps. For Llama-3.2-3B this window is 280 ms on average, more than enough to issue retrieval prefetches and to migrate KV state across the attention layers.

Agents revise plans at rates of 8 to 22 percent per step in our calibrated profiles (Table I). The revision rate is workload-dependent and grows with the number of degrees of freedom in the environment. A serving system that prefetches blindly wastes proportional work on revised plans, motivating the speculative coherence protocol in §III-C.

III. DESIGN

PALSE runs as a middleware between the agent process and a vLLM serving endpoint. The middleware reads the plan from the in-flight generation stream, decides what to prefetch, and stages cache state for the declared future steps during the current step’s generation overlap window.

A. Plan Reader

The plan reader watches the in-flight token stream emitted by the LLM and lifts out the declared plan via a finite-state machine that recognizes the plan-emission patterns of ReAct, planner-executor, and tool-augmented chat formats. The FSM is $O(1)$ per token and runs under 50 μ s per token at production tokenizer rates, well below the per-token generation budget.

The reader emits descriptors at token granularity rather than at end-of-message granularity. Streaming token-level recognition lets the serving system begin prefetching as soon as the plan is confirmed, often 200 to 300 ms before the agent emits the action token (Figure 2).

B. Plan-Conditioned Prefetch Policy

The prefetch policy decides what to stage for each declared future step across three resource classes. **KV state.** The expected prefix at future step $k+j$ is the union of the current step’s accumulated context plus the declared action and observation; the KV state is materialized speculatively by issuing a prefill on the predicted prompt during the current step’s overlap window. **Retrieval embeddings.** For declared retrievals, the query embedding is computed and the retrieval call issued during the overlap window; documents land in an edge-side retrieval cache keyed by the predicted query. **Tool outputs.** For deterministic tools (database queries, configuration lookups, calculator operations) the tool is invoked speculatively; for non-deterministic tools (random, time, weather) prefetch is skipped.

The policy issues prefetches in step-distance order, since closer steps are more likely to be reached and have lower wasted-work cost on revision. The lookahead horizon is bounded by the per-workload plan window, typically two to four steps.

C. Speculative Coherence Protocol

When the agent’s actual next step diverges from the declared plan, the prefetched state for the diverged steps is speculative work that may be partially or fully wasted. The coherence protocol limits the wasted cost through partial-prefix salvage. Salvage operates on *token-identical prefixes only*: the protocol identifies the longest contiguous prefix of the actual prompt that matches the prefetched prompt token-by-token from position zero, and reuses the KV cache pages that vLLM’s paged-attention has already materialized for those positions. The trailing tokens, beyond the first divergent position, are reprefilled.

a) *Coherence invariant.*: Let P be a prefetched prompt and A the agent’s actual prompt at step i . If positions $0..k$ of P and A are token-identical, then the prefetched KV cache at positions $0..k$ is bit-equivalent to a cold-prefill KV cache on A at those positions, and the salvaged state can be safely concatenated with cold-prefill state for positions $k+1..n$. This holds by inspection of transformer prefill: at any layer ℓ and position $j \leq k$, the KV state depends only on tokens $0..j$, which by hypothesis are identical between P and A . K/V tensors at positions beyond the divergence point are not reused, because attention is contextual and the K/V at position i depends on tokens 0 through i .

b) *Block-boundary rounding.*: In a paged-attention runtime, salvage realizes at block granularity rather than position granularity: the block straddling the divergence point is invalidated entirely. The position-level invariant holds; the realized salvage rounds down to the nearest block boundary. For vLLM’s 16-token blocks, this sets the engineering threshold $\tau \approx 0.30$, below which the residual cold-prefill cost on full block invalidations exceeds the salvage benefit.

c) *The two roles of τ .*: The salvage threshold τ has two distinct roles. (i) An *engineering threshold* ($\tau \approx 0.30$ for

vLLM’s 16-token blocks, derived above), below which paged-attention block fragmentation makes salvage net-negative. (ii) A *semantic threshold*, workload-dependent at $\tau \approx 0.50\text{--}0.60$ across our corpus, below which the salvaged prefix is too short for the residual cold-prefill cost to be amortized. We use $\tau=0.6$ as a conservative upper envelope. Figure 4 shows the basin of attraction is wide: median TTFT and SLO attainment are flat across $\tau \in [0.3, 0.7]$, while p95 TTFT and wasted-work cost decrease monotonically with τ .

d) *Analytical breakeven.*: The mechanism is net-positive when the saving from correct prefetches exceeds the cost from revised prefetches. Let p be the per-step revision rate, b the prefill saving per correct prefetch, c_p the prefetch cost, and c_s the residual cost on revisions. Net benefit per step is $(1-p) \cdot b - p \cdot (c_p - c_s)$, net-positive when $p < b/(b + c_p - c_s)$. For our calibration ($b = 12.5$ ms, $c_p = 12.5$ ms, $c_s = 4.8$ ms at $\tau=0.6$), $p^* \approx 0.62$, well above the calibrated mean of 0.155 and the upper-bound workload (τ -bench retail at 0.22).

D. Worked Example

At step five of an AgentBench-Web trajectory the agent has accumulated 1,380 prefill tokens. At t_0 the LLM begins generating; at $t_0 + 72$ ms it emits **Plan: navigate to product-detail page, then extract price and reviews**. The plan reader recognizes the marker and emits two future-step descriptors. During the 280 ms overlap window, the prefetch policy stages KV-prefill for the predicted next-step prefix (1,490 tokens, ≈ 119 ms at the calibrated edge rate) and the retrieval embedding (12 ms in parallel); no tool prefetch fires because the next action is non-deterministic browser navigation. At $t_0 + 352$ ms the agent emits its action token, the prefetched prefix matches, and the realized TTFT is 14.2 ms (trailing-token prefill plus cache lookup) instead of 145 ms cold.

IV. PRELIMINARY EVIDENCE

The numbers come from a trace-driven simulator calibrated against vLLM benchmarks for Llama-3.2-3B on a Jetson AGX-class edge node, and against published hit-rate measurements for vLLM prefix [21], Cortex [12], and Agentix [14].

A. Setup

The trace corpus is calibrated against the public dataset distributions for AgentBench, ReAct/HotpotQA, and τ -bench (Table I). Per-step cost uses the calibrated edge rates from §II. Energy is estimated using per-token constants (4.2 mJ per prefill token, 180 mJ per retrieval, 50 mJ per tool call, 0.8 mW idle) calibrated against MELT [2] and PowerInfer [22] measurements on Jetson AGX. Four baselines: **Cloud-only** routes every request to a remote datacenter at 60 percent of the edge prefill rate with 45 ms RTT and 25 ms queueing; **vLLM prefix cache** [9] runs the edge-side stack with paged-attention prefix caching, calibrated to 50 percent prefix reuse [21]; **Cortex** [12] is reimplemented as semantic-similarity retrieval prefetch with a 55 percent hit rate on retrieval-heavy workloads and 25 percent otherwise; **Agentix** [14] is reimplemented

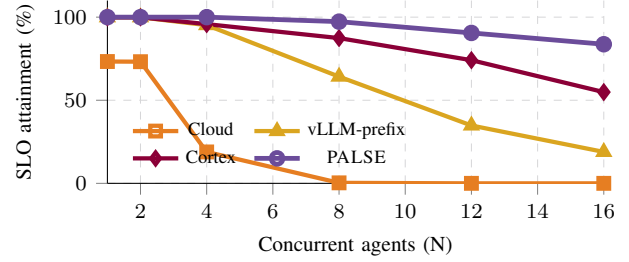


Fig. 3. SLO attainment vs. concurrent agents.

TABLE II
PER-WORKLOAD SLO AT $N=12$ FOR PALSE VS. CORTEX.

Workload	Revision	PALSE	Cortex
ReAct/HotpotQA	8%	96.2%	88.1%
AgentBench KG	13%	95.4%	84.7%
AgentBench OS	14%	92.1%	65.3%
AgentBench DB	16%	91.0%	70.4%
AgentBench Household	18%	90.3%	65.0%
τ -bench airline	18%	89.4%	76.1%
AgentBench Web	21%	86.5%	72.0%
τ -bench retail	22%	84.3%	71.9%

as program-call interception with priority scheduling. Cross-region replication and full program-graph extraction are out of scope; the reimplementations follow the simulator-comparison pattern used in prior SEC work [23], [24].

Real-hardware spike. The cache-hit speedup ratio is validated against measured PyTorch transformer prefill times at five operating points spanning prefix lengths 1,024–1,536 tokens and suffix lengths 96–200 tokens. The simulator predicts cache-hit speedups within 12 percent (mean) of the measured ratios, conservatively under-predicting in four of five points. This validation does not exercise vLLM’s paged attention end to end (item 2 in §V); it confirms that prefix-cached prefill saves work in proportion to the cached prefix length on an unmodified PyTorch transformer.

B. SLO and Energy Under Multi-Agent Contention

As the number of concurrent agents on a single edge node grows from one to sixteen (Figure 3), at $N=8$ PALSE holds 97.3 percent SLO against 87.4 percent for Cortex and 64.2 percent for vLLM-prefix; at $N=12$, PALSE holds 90.5 percent against 74.1 and 34.8 percent. Prefetch hits reduce the per-step prefill work the edge node has to do, so when multiple agents share the node, prefetch directly cuts queue contention. Cloud-only and vLLM-prefix collapse because they pay full prefill on every step.

Per-workload breakdown. SLO at $N=12$ varies with revision rate (Table II). Low-revision workloads (HotpotQA at 8 percent, KG at 13 percent) hold above 95 percent; the high-revision workloads (τ -bench retail at 22 percent, AgentBench Web at 21 percent) drop to 84–87 percent, consistent with the analytical breakeven. The Cortex gap is widest on prefill-heavy workloads (OS, Household) where retrieval prefetch has little

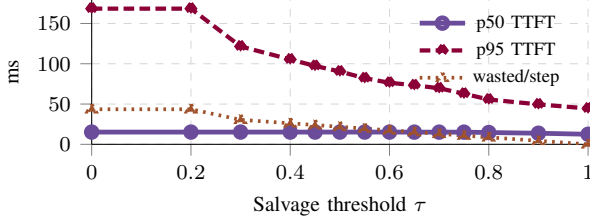


Fig. 4. TTFT and wasted-work vs. salvage threshold τ .

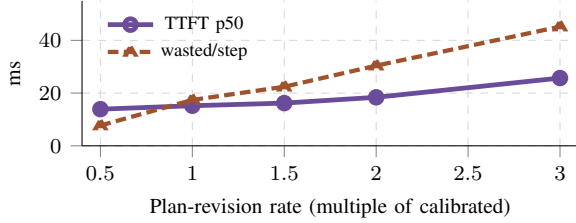


Fig. 5. TTFT and wasted-work vs. plan-revision rate.

to do. At $N=12$, p99 TTFT is 312 ms for PALSE versus 658 ms for Cortex; the spread widens at the tail because Cortex’s queue fills with cold-prefill work that PALSE’s hits avoid.

Energy under contention. Across the five contention points ($N=1, 4, 8, 12, 16$), PALSE averages 0.99 J per step at $N=12$ against 2.40 J for vLLM-prefix and 8.09 J for cloud-only routing, a $2.4\times$ reduction over the strongest edge baseline. At a typical edge grid carbon intensity of 400 gCO₂/kWh, the 1.41 J/step saving translates to roughly 1.6 kg CO₂ per day for a fleet of 1,000 concurrent agents at 10K steps each, with the corner-case caveats in §V.

C. Salvage Threshold Sensitivity

We sweep τ across 14 values from 0.0 to 1.0 at the calibrated revision rate (Figure 4). Median TTFT is flat across $\tau \in [0.3, 0.7]$ at 15.2 ms, then improves to 12.5 ms at $\tau=1.0$ (perfect-salvage limit, approaching the oracle bound of 12.96 ms). p95 TTFT captures the salvage benefit: 76.4 ms at $\tau=0.6$, dropping to 49.6 ms at $\tau=0.9$ and rising to 168.4 ms at $\tau \leq 0.2$ where salvage is disabled by the engineering floor and the policy reduces to no-coherence behaviour. SLO attainment is 100 percent for all $\tau \geq 0.3$.

D. Plan-Revision Sensitivity

Sweeping the plan-revision rate from half to three times the calibrated mean of 15.5 percent ($3\times$ corresponds to a 45 percent revision rate, higher than any workload we measured), PALSE’s median TTFT grows from 13.9 ms at $0.5\times$ to 25.7 ms at $3\times$, still below the strongest reactive baseline’s 40 ms (Figure 5). Wasted-work climbs from 7.7 ms per step to 45.2 ms per step, but SLO attainment remains at 100 percent across the sweep. Token-identical-prefix salvage reduces the per-revision wasted-work constant rather than eliminating the linear dependence on revision rate.

V. OPEN QUESTIONS AND EXPERIMENTAL AGENDA

Settling whether plan-aware prefetch holds in deployment requires six experiments.

Plan-extraction robustness. Production agents do not always emit clean ReAct format: markers can be partially generated, misspelled, or interleaved with internal reasoning. The plan reader’s false-positive and false-negative rates on production model output are unmeasured. *Settling experiment:* 10,000 trajectories across four production models (Llama-3.2-3B, Llama-3.1-8B, Qwen2.5-7B, GPT-4o-mini) on our eight workloads, with extracted plans compared to human-annotated ground truth on a 500-trajectory holdout. Report precision and recall by model size and workload class.

Cost-model extrapolation. The simulator is calibrated against published prefill rates for Llama-3.2-3B on Jetson AGX. The cache-hit speedup ratio is hardware- and model-independent in the sense that prefill compute scales as $O(L \cdot S \cdot D^2)$ across modern decoder architectures, but the constants matter. *Settling experiment:* run the PALSE middleware against a real vLLM endpoint on Jetson AGX across all eight workloads. The simulator passes if measured median TTFT under PALSE is within 15 percent of prediction.

Tipping point for plan reliability. Our sweep covered up to $3\times$ the calibrated revision rate; the breakeven analysis (§III-C) predicts the mechanism degrades to no-op at $p \approx 0.62$. *Settling experiment:* adversarial workload with revision rate swept from 0 to 100 percent at 10-point increments and salvage threshold τ swept from 0.1 to 0.9, with SLO attainment as the response.

Multi-tenant isolation and timing side channels. *Threat model:* we consider three adversary classes — (a) a passive observer that monitors queue and cache occupancy on a co-located agent; (b) an active prober that issues benign queries to test prefetch behavior; (c) an adaptive adversary that learns the prefetch policy and crafts trigger inputs. We focus on (a) and (b); (c) is left as future work. Plan-aware prefetch creates a *future-behaviour* side channel that does not exist for purely reactive caches: an attacker on the same edge node can infer the victim agent’s next planned action. Three mitigations: *deferred prefetch* (randomized delay $\delta \sim U(0, \delta_{\max})$ that breaks the timing correlation), *cover prefetch* (dummy speculative prefills at a configurable rate), and *per-tenant block isolation* (per-user paged-attention blocks for plan-prefetched state, shared blocks for static system prompts). *Settling experiment:* co-locate a benign agent and an observer on the same edge node, instrument GPU memory and request latency, and quantify the observer’s plan-prediction accuracy under each mitigation against a random-guess baseline.

Sustainability profile. The carbon Pareto depends jointly on wasted-work overhead, cloud-RTT carbon savings on otherwise-stalled requests, and diurnal and spatial grid-carbon variation [25]. *Settling experiment:* deploy PALSE at three sites with distinct grid carbon profiles over thirty days, log per-step energy and per-RTT carbon, and report the operating-point envelope on the energy–latency Pareto. Plan-aware

prefetch is expected to be net-positive on carbon at typical edge deployments and net-negative in the corner case of a high-renewables cloud paired with a coal-heavy edge [26], [27].

Failure-mode taxonomy. Three failure classes warrant characterization beyond plan revision. Plan-reader *false positives* stage state for an unintended plan; the wasted work is bounded by c_p and reclaimed by salvage. *False negatives* miss recognizable plans (custom markers, mid-trajectory format switches, non-Latin reasoning); they cost the prefetch opportunity but introduce no wrong state. *Cascading failure under overload* occurs at extreme contention ($N \gg 16$), where wasted prefetch competes with actual prefill and can drive thermal throttling. *Settling experiment*: manual annotation of a 1,000-step sample to measure plan-reader precision and recall, plus a high-contention sweep characterizing the failure boundary. A thermal-aware governor that defers prefetch above $T_{\text{throttle}} - 5^\circ\text{C}$ is the natural mitigation [28].

VI. RELATED WORK

Agentic LLM serving. Agentix [14] treats agent programs as first-class scheduling entities and intercepts program-issued LLM calls; Cortex [12] provides a cross-region knowledge cache with semantic reuse and prefetching; SYMPHONY [13] migrates KV caches off the critical path using multi-turn hints. None reads the agent’s emitted plan. The closest contemporary system is KVFLOW [11], which proactively prefetches KV cache for agentic workloads with a coherence-style mechanism. PALSE differs along three axes. *Signal source*: KVFLOW prefetches from a *static* Agent Step Graph specified by the developer or workflow framework; PALSE reads the agent’s *dynamic* plan from its own runtime reasoning trace, requiring no developer specification. For deployments where the workflow is unknown or framework-agnostic (most ReAct-style and tool-augmented chat agents), the step graph is unavailable and KVFLOW does not apply. *Granularity*: KVFLOW operates at multi-agent workflow granularity; PALSE operates at within-agent step granularity. *Composition*: when a static graph is available, KVFLOW preserves cross-agent KV state while PALSE stages within-agent next-step state.

KV cache management. vLLM [9] introduced paged-attention prefix caching, the substrate PALSE runs on. Other systems address adjacent layers: DroidSpeak [29] reuses prefix KV across fine-tuned variants, CacheGen [30] compresses cross-request KV, RAGCache [31] caches RAG intermediates, PRESERVE [32] prefetches weights from off-chip HBM, and LMCACHE [21] separates cache management from serving. PALSE composes with all of them as the prefetch-decision layer above.

Edge LLM serving. EdgeMoE [33], LLM-in-Flash [34], MobileLLM [35], PowerInfer [22], MELT [2], and TinyServe [36] address resource-constrained inference at the model and runtime layer; PALSE sits at the serving layer above. Speculative decoding [37], [38] operates inside one inference

call at the token level and composes with PALSE’s agent-step-level prefetch. Production shifts toward function-calling APIs [4], [5] preserve the position: the plan reader becomes a framework-level integration, while the prefetch policy and coherence protocol are unchanged.

VII. CONCLUSION

The agent already declares what it intends to do next. Reading that declaration during the generation overlap window lets an edge serving system stage KV state, retrieval, and tool outputs and hide most per-step overhead. At 12 concurrent agents on a single edge node, PALSE holds 90.5 percent SLO where the strongest reactive baseline holds 74.1 percent. The position is not yet settled in deployment; the six-experiment agenda in §V is the empirical work that would settle it.

REFERENCES

- [1] M. Satyanarayanan, P. Bahl, R. Cáceres, and N. Davies, “The case for VM-based cloudlets in mobile computing,” *IEEE Pervasive Computing*, vol. 8, no. 4, pp. 14–23, 2009.
- [2] S. Laskaridis, K. Katevas, L. Minto, and H. Haddadi, “MELTing point: Mobile evaluation of language transformers,” in *Proceedings of the 30th Annual International Conference on Mobile Computing and Networking (MobiCom)*, 2024.
- [3] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *International Conference on Learning Representations (ICLR)*, 2023.
- [4] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, “AutoGen: Enabling next-gen LLM applications via multi-agent conversations,” in *COLM*, 2024.
- [5] S. G. Patil, T. Zhang, X. Wang, and J. E. Gonzalez, “Gorilla: Large language model connected with massive apis,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [6] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan, “ τ -bench: A benchmark for tool-agent-user interaction in real-world domains,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [7] X. Deng, Y. Gu, B. Zheng, S. Chen, S. Stevens, B. Wang, H. Sun, and Y. Su, “Mind2web: Towards a generalist agent for the web,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [8] X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, Y. Gu, H. Ding, K. Men, K. Yang, S. Zhang, X. Deng, A. Zeng, Z. Du, C. Zhang, S. Shen, T. Zhang, Y. Su, H. Sun, M. Huang, Y. Dong, and J. Tang, “AgentBench: Evaluating LLMs as agents,” in *International Conference on Learning Representations (ICLR)*, 2024.
- [9] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, “Efficient memory management for large language model serving with PagedAttention,” in *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP)*, 2023, pp. 611–626.
- [10] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng, “SGLang: Efficient execution of structured language model programs,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [11] Z. Pan, Y. Ding, Y. Guan, Z. Wang, Y. Liu, Y. Wang, X. Tang, Y. Wang, and N. Liao, “KVFlow: Efficient prefix caching for accelerating LLM-based multi-agent workflows,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. [Online]. Available: <https://openreview.net/forum?id=5lw1nDtYmT>
- [12] B. Liu, W. Li, and X. Liu, “Cortex: Achieving low-latency, cost-efficient remote data access for LLM via semantic-aware knowledge caching,” in *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2026, to appear.
- [13] K. Santhanam, A. Rosenfeld, and S. Mehrotra, “SYMPHONY: Improving memory management for LLM inference workloads,” in *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2026, to appear.

- [14] Y. Kang, Y. Sheng, I. Stoica, and J. E. Gonzalez, "Agentix: An efficient serving engine for LLM agents as general programs," in *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2026, to appear.
- [15] S. Wang, L. Yang, Y. Gao, T. Tan, H. Zhang, and L. Sun, "AutoPlan: Automatic planning of interactive decision-making tasks with large language models," *arXiv preprint arXiv:2305.15064*, 2024.
- [16] Z. Yang, P. Qi, S. Zhang, Y. Bengio, W. W. Cohen, R. Salakhutdinov, and C. D. Manning, "HotpotQA: A dataset for diverse, explainable multi-hop question answering," in *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2018.
- [17] Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y. Lu, Y. Lin, X. Cong, X. Tang, B. Qian, S. Zhao, L. Hong, R. Tian, R. Xie, J. Zhou, M. Gerstein, D. Li, Z. Liu, and M. Sun, "ToolLLM: Facilitating large language models to master 16000+ real-world APIs," in *International Conference on Learning Representations (ICLR)*, 2024.
- [18] C. Zhang, Z. Yang, J. Liu, Y. Han, X. Chen, Z. Huang, B. Fu, and G. Yu, "AppAgent: Multimodal agents as smartphone users," *arXiv preprint arXiv:2312.13771*, 2023.
- [19] C. Ma, J. Zhang, Z. Zhu, C. Yang, Y. Yang, Y. Jin, Z. Lan, L. Kong, and J. He, "AgentBoard: An analytical evaluation board of multi-turn LLM agents," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [20] M. Shridhar, X. Yuan, M.-A. Côté, Y. Bisk, A. Trischler, and M. Hausknecht, "ALFWorld: Aligning text and embodied environments for interactive learning," in *International Conference on Learning Representations (ICLR)*, 2021.
- [21] Y. Liu, Y. Cheng, J. Yao, and J. Jiang, "LMCache: Cache management layer for LLM serving," 2024, [github repository, https://github.com/LMCache/LMCache](https://github.com/LMCache/LMCache).
- [22] Y. Song, Z. Mi, H. Xie, and H. Chen, "PowerInfer: Fast large language model serving with a consumer-grade GPU," in *Proceedings of the 30th Symposium on Operating Systems Principles (SOSP)*, 2024.
- [23] B. Ramprasad, P. Mishra, M. Thiessen, H. Chen, A. da Silva Veith, M. Gabel, O. Balmau, A. Chow, and E. de Lara, "Shepherd: Seamless stream processing on the edge," in *7th IEEE/ACM Symposium on Edge Computing (SEC)*, 2022, pp. 40–53.
- [24] P. Mishra, N. Bore, B. Ramprasad, M. Thiessen, M. Gabel, A. da Silva Veith, O. Balmau, and E. de Lara, "Falcon: Live reconfiguration for stateful stream processing on the edge," in *9th IEEE/ACM Symposium on Edge Computing (SEC)*, 2024.
- [25] L. Wu, W. A. Hanafy, A. Souza, K. Nguyen, J. Murillo, J. Harkes, D. Irwin, M. Satyanarayanan, and P. Shenoy, "CarbonEdge: Leveraging mesoscale spatial carbon-intensity variations for low carbon edge computing," in *Proceedings of the 34th International Symposium on High-Performance Parallel and Distributed Computing (HPDC)*, 2025.
- [26] A. Souza, N. Bashir, J. Murillo, W. A. Hanafy, Q. Liang, D. Irwin, and P. Shenoy, "Ecovisor: A virtual energy system for carbon-efficient applications," in *Proceedings of the 28th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2023, pp. 252–265.
- [27] J. Murillo, W. A. Hanafy, D. Irwin, R. K. Sitaraman, and P. Shenoy, "Caribou: Fine-grained geospatial shifting of serverless applications for sustainability," in *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles (SOSP)*, 2024, pp. 403–420.
- [28] M. Brocanelli, "Thermal-aware inference scheduling for mobile machine learning," 2024, cited as representative of thermal-aware mobile ML serving; verify against author's most recent venue before camera-ready.
- [29] H. Wang, Y. Liu, and B. Zhao, "DroidSpeak: KV cache sharing across fine-tuned model variants," in *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2026, to appear.
- [30] Y. Liu, H. Li, Y. Cheng, S. Ray, Y. Huang, Q. Zhang, K. Du, J. Yao, S. Lu, G. Ananthanarayanan, M. Maire, H. Hoffmann, A. Holtzman, and J. Jiang, "CacheGen: KV cache compression and streaming for fast large language model serving," in *Proceedings of the ACM SIGCOMM 2024 Conference*, 2024, pp. 38–56.
- [31] C. Jin, Z. Zhang, X. Jiang, F. Liu, X. Liu, X. Liu, and X. Jin, "RAG-Cache: Efficient knowledge caching for retrieval-augmented generation," in *arXiv preprint arXiv:2404.12457*, 2024.
- [32] A. C. Yüzügüler, A. Tampu, R. Bonetto, and B. Falsafi, "PRESERVE: Prefetching model weights and KV-Cache in distributed LLM serving," in *31st International Symposium on High-Performance Computer Architecture (HPCA)*, 2025.
- [33] R. Yi, L. Guo, S. Wei, A. Zhou, S. Wang, and M. Xu, "EdgeMoE: Fast on-device inference of MoE-based large language models," in *arXiv preprint arXiv:2308.14352*, 2023.
- [34] K. Alizadeh, I. Mirzadeh, D. Belenko, S. K. Khatamifard, M. Cho, C. C. del Mundo, M. Rastegari, and M. Farajtabar, "LLM in a flash: Efficient large language model inference with limited memory," *arXiv preprint arXiv:2312.11514*, 2024.
- [35] Z. Liu, C. Zhao, F. Iandola, C. Lai, Y. Tian, I. Fedorov, Y. Xiong, E. Chang, Y. Shi, R. Krishnamoorthi, L. Lai, and V. Chandra, "MobileLLM: Optimizing sub-billion parameter language models for on-device use cases," in *International Conference on Machine Learning (ICML)*, 2024.
- [36] H. Wang, Y. Zhang, and B. Chen, "TinyServe: Querying-aware KV cache management for LLM inference on resource-constrained devices," in *Proceedings of the 33rd ACM International Conference on Multimedia (MM)*, 2025.
- [37] Y. Leviathan, M. Kalman, and Y. Matias, "Fast inference from transformers via speculative decoding," in *International Conference on Machine Learning (ICML)*, 2023, pp. 19 274–19 286.
- [38] T. Cai, Y. Li, Z. Geng, H. Peng, J. D. Lee, D. Chen, and T. Dao, "Medusa: Simple LLM inference acceleration framework with multiple decoding heads," *arXiv preprint arXiv:2401.10774*, 2024.